Clase 8 — El RAG se muda

Apunte del alumno · Desarrollo con IA · Postgres + pgvector, embeddings en tu compu y tu propio corpus

El RAG de las clases 6 y 7 funciona, pero guarda los vectores en un archivo, le pide cada embedding a Google y contesta sobre un reglamento que no elegiste. En esta clase cambia de casa tres veces: los vectores pasan a una base de datos, el modelo de embeddings pasa a tu compu y el reglamento se cambia por tu corpus. Al terminar tenés el sistema que vas a presentar en las exposiciones.

Parte	Cuándo se usa
Parte 1 — Fichas de trabajo	Durante la clase, junto con la guía Hacelo vos. Se completan a mano.
Parte 2 — Conceptos y definiciones	Después de la clase. Material de estudio.

Al terminar la clase vas a tener

- Tu RAG funcionando sobre tu corpus, con los vectores en Postgres y los embeddings calculados en tu compu.
- Una marca que reconoce la estructura de tu documento.
- Un umbral calibrado para tu corpus, con la recta que lo justifica.
- Cuatro preguntas probadas, con su fuente y su diagnóstico: el material de tu exposición.

> **ATENCIÓN**
>
> La Parte 2 explica lo que vas a descubrir en las fichas. Leela después de la clase: si la leés antes, las predicciones de las fichas pierden sentido.

Parte 1 — Fichas de trabajo

Cada ficha corresponde a un paso de la guía Hacelo vos. Las que dicen «se conserva» van a tu portafolio y las vas a usar en la exposición.

Ficha 1 — La dimensión (paso 3)

Antes de correr prueba.mjs

En las clases 6 y 7 cada embedding tenía 1536 números. ¿Cuántos creés que va a devolver bge-m3? ¿Por qué?

Después de correr

¿Cuántos devolvió? ¿En qué dos lugares del proyecto tenés que escribir ese número?

Ficha 2 — Tu estructura (paso 5)

	Tu respuesta
La marca que escribiste	
Fragmentos que salieron	
Fragmentos con sección reconocida	
El más corto y el más largo (caracteres)	

¿Las secciones son las que esperabas? Si alguna parte quedó sin reconocer, ¿qué tiene de distinto su comienzo?

Si el más largo pasa de 2000 caracteres, ¿qué le hace la función? ¿Qué se pierde?

Si tu marca es null: ¿por qué tu corpus no tiene una marca que sirva? ¿Qué problema de la clase 7 vuelve a aparecer?

Ficha 3 — Indexar otra vez (paso 7)

Antes de correr

Filas en la tabla después del paso 6: ……. Si corrés indexar.mjs otra vez sin cambiar nada, ¿cuántas filas va a haber? ¿Por qué?

Después de correr

Filas que hay: ……. ¿Acertaste? Si le hicieras una pregunta al sistema ahora, ¿qué verías entre los tres primeros resultados? ¿Por qué en las clases 6 y 7 esto no pasaba nunca?

Ficha 4 — Dónde cortar (paso 9) · se conserva

Tu UMBRAL de la clase 7: ……. ¿Te sirve para hoy? ¿Por qué?

Anotá la similitud del mejor fragmento para cada pregunta que imprime distribucion.mjs.

Pregunta	¿Está?	Similitud
	sí	
	sí	
	sí	
	sí	
	no	
	no	
cuál es la contraseña del wifi	no	
quién ganó el último mundial	no	
cómo se hace una milanesa	no	

Marcá cada pregunta en la recta: una cruz para las que están, un círculo para las que no. Marcá también tu umbral de la clase 7 con una flecha.

0	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9	1

1. Con el umbral de la clase 7, ¿cuántas preguntas que están se descartarían? ¿Cuántas que no están pasarían?
2. ¿Hay un hueco limpio entre cruces y círculos, o se pisan?
3. Tu umbral nuevo: ……
4. Si se pisan, ¿qué preguntas quedan del lado equivocado? ¿Por qué creés que les pasa?

Ficha 5 — Tus cuatro preguntas (paso 10) · se conserva para la exposición

#	Similitud del primero	Fuente que citó	¿Correcta?	Si falló: ¿búsqueda o modelo?
1				
2				
3				
4				

> Para saber quién falló, corré la pregunta con --sin-modelo. Si el fragmento correcto no está entre los tres, falló la búsqueda. Si está y la respuesta igual es mala, falló el modelo.

Para la exposición

Elegí y anotá: una pregunta que funciona, una que no está y el sistema dice que no, y una que falla, con tu explicación de por qué.

Parte 2 — Conceptos y definiciones

> **ATENCIÓN**
>
> Leela después de la clase.

1. Las tres mudanzas

El sistema de esta clase tiene las mismas piezas que el de la clase 7: troceado, embeddings en lote, búsqueda por similitud, umbral, prompt y respuesta. Cambian tres cosas de lugar. Los vectores pasan de un archivo JSON a una base de datos. El modelo de embeddings pasa de la API de Google a la propia compu, con Ollama. Y el documento común se reemplaza por un corpus propio. La generación de la respuesta no se muda: la sigue haciendo un modelo de Google por API.

Esa arquitectura, recuperar fragmentos de una fuente propia y pasárselos al modelo antes de que responda, es la que se publicó con el nombre de generación aumentada con recuperación, o RAG[1].

2. Por qué una base de datos

Un archivo como indice.json tiene cuatro limitaciones cuando el corpus crece: se lee entero en cada pregunta aunque se usen tres fragmentos; lo puede usar un solo programa por vez; no permite pedir una parte (los fragmentos de un archivo, por ejemplo) sin leer y reescribir todo; y no sabe buscar, porque la búsqueda la hace el programa comparando contra todos.

> **DEFINICIÓN — base de datos**
>
> Programa que guarda datos de forma persistente y responde consultas sobre ellos, aunque muchos programas le hablen al mismo tiempo. Los datos se piden con un lenguaje de consultas, y la base se encarga de encontrarlos sin que el programa tenga que leer todo.

Postgres es una base de datos relacional: organiza los datos en tablas, cada fila es un elemento y cada columna una propiedad con un tipo fijo. El modelo relacional fue propuesto por Codd en 1970[2] y se consulta con SQL. En el RAG de esta clase, cada objeto del índice de la clase 7 es una fila de la tabla fragmentos, y cada metadato (archivo, sección, posición) es una columna.

Persistencia

Una base guarda todo lo que recibe hasta que se le indique lo contrario. Por eso, a diferencia de writeFile, que reemplaza el archivo entero, un INSERT agrega: indexar dos veces duplica cada fragmento. La solución de la clase es vaciar la tabla antes de indexar. Los sistemas grandes hacen actualización incremental, que reemplaza solo los fragmentos de los documentos que cambiaron.

[1] Lewis et al. (2020) presentaron la técnica con ese nombre, combinando un recuperador de fragmentos con un modelo generador.

[2] Codd (1970) propuso representar los datos como relaciones (tablas) independientes de cómo se almacenan físicamente.

Consultas parametrizadas

Los valores de una consulta se pasan como $1, $2… y viajan separados de la orden. Si se pegaran adentro del texto de la consulta, un dato con un apóstrofo la rompería, y un dato escrito a propósito podría agregar órdenes propias: ese ataque se llama inyección SQL.

3. pgvector

> **DEFINICIÓN — pgvector**
>
> Extensión de Postgres que agrega un tipo de columna para guardar vectores, las operaciones para medir la distancia entre ellos, y los índices para buscar los más cercanos sin compararlos a todos.

Que sea una extensión y no una base aparte permite tener en la misma fila el texto, los metadatos y el vector. La columna se declara VECTOR(n), donde n es la cantidad exacta de números que acepta: un vector de otra dimensión se rechaza con un error.

Distancia y similitud

pgvector mide la distancia coseno, que es uno menos la similitud coseno de la clase 6: 0 cuando dos vectores apuntan para el mismo lado y valores más grandes cuanto más se separan. El operador es <=>. Buscar los k fragmentos más parecidos es ordenar por distancia de menor a mayor y quedarse con los primeros k:

CÓDIGO — SQL

```
SELECT id, texto, 1 - (embedding <=> $1) AS similitud
  FROM fragmentos
 ORDER BY embedding <=> $1
 LIMIT 3
```

La similitud se calcula como 1 menos la distancia para que el umbral y los números se lean igual que en las clases 6 y 7. El orden, en cambio, se pide por la distancia: es la forma en que Postgres puede usar el índice.

4. Buscar sin mirar a todos: HNSW

La búsqueda de la clase 6 es exhaustiva: compara la pregunta contra cada fragmento. Siempre encuentra el más cercano, pero el trabajo crece en proporción al corpus. Los índices de búsqueda aproximada de vecinos evitan esa comparación completa. El que usa la clase es HNSW, sigla de Hierarchical Navigable Small World[3].

Primera idea: cada punto conoce a sus vecinos

Al construir el índice, cada vector se conecta con unos pocos vectores cercanos. El resultado es un grafo, una red de puntos unidos, con algunas conexiones largas además de las cortas.

[3] Malkov y Yashunin (2020) describen el algoritmo, su estructura de niveles y su comparación con otros métodos de búsqueda aproximada.

Segunda idea: buscar es saltar de vecino en vecino

La búsqueda parte de un punto fijo, evalúa a sus vecinos y pasa al que está más cerca de la pregunta. Repite hasta que ningún vecino mejora. Solo compara contra los vecinos de los puntos del camino.

Tercera idea: los pisos

El grafo tiene varios niveles. Los de arriba tienen pocos puntos y conexiones largas; el de abajo, todos los puntos y conexiones cortas. La búsqueda empieza arriba para acercarse rápido a la zona de la pregunta y baja de nivel para afinar.

Cuarta idea: es aproximado

El camino puede detenerse en un punto cercano que no es el más cercano de todos. HNSW cambia una pequeña pérdida de exactitud por una gran ganancia de velocidad.

> **DEFINICIÓN — búsqueda aproximada**
>
> Búsqueda que puede no devolver el elemento más cercano de todos a cambio de ser mucho más rápida.

> Con un corpus de decenas o cientos de fragmentos la diferencia de velocidad no se nota. El índice importa cuando el corpus llega a cientos de miles o millones de fragmentos.

5. El modelo de embeddings en la propia compu

El modelo de embeddings de la clase es bge-m3[4], que se corre localmente con Ollama. Tiene alrededor de 570 millones de parámetros, devuelve vectores de 1024 números y está entrenado con textos en muchos idiomas, español incluido.

Un modelo de embeddings recorre el texto una sola vez y devuelve un vector. Un modelo de chat, en cambio, genera la respuesta token por token con una pasada por cada token, y los que escriben bien tienen miles de millones de parámetros. Por eso el primero corre en una compu común y el segundo, en la práctica, necesita los servidores de un proveedor.

El reparto también conviene por la cantidad de pedidos. Indexar hace un pedido por lote sobre todo el corpus, y se repite cada vez que cambia el troceado; preguntar hace un solo pedido de embedding y uno al modelo de chat. Con los embeddings en la compu, la parte que hace más pedidos no consume el cupo del proveedor.

Para comparar modelos de embeddings existen benchmarks públicos, como MTEB[5]. Igual que cualquier benchmark, sirve de referencia: lo que decide es cómo funciona el modelo con el corpus propio.

[4] Chen et al. (2024) presentan el modelo, entrenado para más de cien idiomas y para textos de hasta 8192 tokens.

[5] Muennighoff et al. (2023) proponen un benchmark que evalúa modelos de embeddings en decenas de tareas y conjuntos de datos.

6. Dos trampas

La dimensión de la columna

VECTOR(n) acepta exactamente n números. Si el modelo devuelve otra cantidad, la base rechaza el dato con un error del tipo «expected 1536 dimensions, not 1024». Es una protección: el número del .env y el de la columna tienen que coincidir siempre.

Mismo tamaño, otro mapa

Dos modelos distintos ubican los textos en espacios distintos, aunque se les pida la misma cantidad de números. Si el corpus se indexa con un modelo y la pregunta se convierte con otro, la dimensión coincide, no hay ningún error, y los resultados no significan nada. La regla es usar el mismo modelo para el corpus y para la pregunta.

7. El umbral se calibra por modelo

Los valores de similitud dependen del modelo: en uno, dos textos sin relación pueden dar 0,5 y en otro 0,3. Lo que se mantiene es el orden dentro de cada modelo. Por eso el umbral de la clase 7, medido con el modelo de Google, no sirve con bge-m3, y por eso se vuelve a calibrar con preguntas cuya respuesta se conoce. Cambiar el modelo, el corpus o el troceado obliga a volver a medir.

8. Contenedores

La base corre adentro de Docker. Un contenedor es un programa que corre aislado del resto del sistema, con sus propios archivos y configuración, y se crea a partir de una imagen, un paquete con el programa y todo lo que necesita. Esto permite que todos los participantes de un proyecto usen exactamente el mismo entorno, sin importar su sistema operativo[6]. Los datos de la base se guardan en un volumen, que sobrevive aunque el contenedor se apague o se borre.

9. Lo que quedó afuera

El RAG de la materia tiene las piezas esenciales. Los sistemas de producción suelen sumar cuatro más:

- Reranking: después de traer muchos fragmentos con el índice, un modelo más preciso y más lento los vuelve a ordenar mirando la pregunta y cada fragmento juntos.
- Búsqueda híbrida: combinar la búsqueda por significado con una búsqueda por palabras, como BM25[7], útil para códigos, nombres propios y números.
- Actualización incremental: reindexar solo los documentos que cambiaron.
- Medición formal: un conjunto grande de preguntas con su respuesta correcta para medir, en porcentaje, cuántas veces el fragmento correcto aparece entre los primeros.

[6] Boettiger (2015) analiza el uso de Docker para que un trabajo pueda reproducirse con el mismo entorno en distintas computadoras.

[7] Robertson y Zaragoza (2009) desarrollan el modelo probabilístico de relevancia en el que se basa BM25.

Glosario

Término	Definición
Base de datos	Programa que guarda datos de forma persistente y responde consultas, aunque muchos programas le hablen a la vez.
Tabla, fila, columna	La tabla agrupa datos de un tipo; cada fila es un elemento; cada columna, una propiedad con tipo fijo.
SQL	Lenguaje para crear tablas y consultar o modificar datos en una base relacional.
Extensión	Agregado que suma a Postgres tipos, operaciones e índices que no trae de fábrica.
pgvector	Extensión de Postgres para guardar vectores, medir distancias e indexarlos.
VECTOR(n)	Tipo de columna que guarda exactamente n números.
Distancia coseno	Uno menos la similitud coseno. En pgvector, el operador <=>.
Búsqueda exhaustiva	Comparar la pregunta contra todos los elementos.
Búsqueda aproximada	Búsqueda que puede no devolver el más cercano a cambio de ser mucho más rápida.
HNSW	Índice que busca saltando de vecino en vecino en un grafo de varios niveles.
Persistencia	Que los datos sigan disponibles después de que el programa que los escribió terminó.
Consulta parametrizada	Consulta con $1, $2… donde los datos viajan separados de la orden.
Inyección SQL	Ataque que mete órdenes dentro de un dato para que la base las ejecute.
Contenedor, imagen, volumen	Programa aislado; paquete del que se crea; lugar donde guarda datos que sobreviven.
Marca	Expresión regular que reconoce el párrafo donde empieza cada parte de un documento.
Actualización incremental	Reindexar solo lo que cambió.
Reranking	Reordenar con un modelo más preciso los fragmentos recuperados.
Búsqueda híbrida	Combinar búsqueda por significado con búsqueda por palabras.

Bibliografía

Boettiger, C. (2015). An introduction to Docker for reproducible research. ACM SIGOPS Operating Systems Review, 49(1), 71–79. https://doi.org/10.1145/2723872.2723882

Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., & Liu, Z. (2024). M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. En Findings of the Association for Computational Linguistics: ACL 2024 (pp. 2318–2335). https://doi.org/10.18653/v1/2024.findings-acl.137

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387. https://doi.org/10.1145/362384.362685

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459–9474.

Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). MTEB: Massive text embedding benchmark. En Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (pp. 2014–2037). https://doi.org/10.18653/v1/2023.eacl-main.148

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval, 3(4), 333–389. https://doi.org/10.1561/1500000019